C1 网页编辑与确定性导出 Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (☐) syntax for tracking.

Goal: 交付可在本地启动的网页相册编辑器，从照片导入、图层编辑和保存恢复，到冻结文档、预检及实际 PNG／JPG／普通 PDF 导出。

Architecture: 自有文档模型是事实源，Fabric 只是画布适配器。网页与可信 headless 入口共用资源、滤镜、文字布局和渲染包；先实现本地仓库端口，后续 C3 替换为云端仓库，C4 接入持久化任务与权益。

Tech Stack: Node 24 LTS、npm workspaces、TypeScript、React、Vite、Fabric、IndexedDB、Vitest、Playwright 固定 Chromium、pdf-lib、fflate；精确候选版本见附录 A，实际安装后生成 lockfile。

Spec: v0.3 基线、PRD、架构、33 配方、测试运维。其他单元归属见交付路线。

状态：待评审的实施计划；下列测试代码与命令是实施时要执行的步骤，本次没有执行产品测试，也没有产品功能通过记录。用户确认全程 AI 开发；不按人工人数估算交期。

Global Constraints

- 文档坐标 px96，schemaVersion=1，colorSpace=srgb；单页为 pageSize，跨页宽度为单页两倍；封面正背面各为单页，另计。

- 首期草案允许 1 – 50 个内页跨页，即 2 – 100 个内页；封面正面、背面另存，不计内页数量。

- 单项目 500 张照片、单图 25MB、100 个图层／跨页；首期单图不超过 80MP。它们是待目标设备验证的容量上限，不是已通过的性能指标。

- 首期支持 JPEG、PNG；HEIC、RAW、PSD 暂不接受；不开放任意 SVG 上传。

- Group 禁止循环、重复 ID、NaN／Infinity 和超深嵌套，深度最多 5；图层数组从底到顶。

- 图片先校正 EXIF 方向，crop 为校正后源图的归一化矩形，范围必须位于 [0,1]，非空；原片不覆盖。

- 原图单列；F01 – F33 稳定 ID；强度 0 – 100%；配方顺序固定；强度在最终效果与原图的线性光域混合；seed 固定。

- 文档、素材、字体、模板和滤镜版本可追溯；缺原片不能回退缩略图；字体缺失／无授权、未知 schema、文字超框或不支持字符阻断成品导出。

- 裁剪后有效 PPI 100 – 199 警告；小于 100 强警告，普通数字导出可明确确认继续；不能靠 DPI 标签提升细节。

- 导出像素＝逻辑 px × DPI ÷96；600×300mm／300DPI 裁切区 7087×3543px；各边 3mm 出血后 7157×3614px。

- 普通 PDF 为 sRGB 栅格页，不承诺 PDF/X、CMYK、可搜索矢量文字、拼版或专业印厂交付。

- 网页完整编辑、小程序快速成册、App 预留；C1 仅交付网页与共享核心，其他首期模块仍在 C2 – C5。

Review Focus

- RF1：未知图层／扩展字段经过读取、局部编辑、保存后仍完整；未知层不能被静默省略导出。测试归属任务 1、8。

- RF2：IndexedDB 配额不足、并发标签页和保存冲突不毁坏上一版本，不把失败显示为“已保存”。测试归属任务 3。

- RF3：EXIF 6／8、透明 PNG、裁剪越界和极端长宽比不会误转方向、改变原片或虚报 PPI。测试归属任务 2、8。

- RF4：中文标点、组合字符、缺字与字体加载竞态不会以系统字体替代后悄悄导出。测试归属任务 2、5、8。

- RF5：React StrictMode、切换跨页、迟到图片加载和取消不会修改新页面或泄漏画布／对象 URL。测试归属任务 5、6、9。

交付边界与统一接口

C1 可在单用户本地浏览器完成完整闭环，界面显示“仅此浏览器保存”，提供可恢复的项目包。本地导出进度可取消，关闭页面不会假装仍有后台任务。正式登录、云存储、可靠导出队列和会员验证由 C3／C4 实现。C1 不调用生图、不展示虚构支付成功、不提供可绕过的假会员。

以下为本计划新增、待评审的工程决定，实施后不得暗改：

- MAX_IMPORT_BYTES=25_000_000，MAX_SOURCE_PIXELS=80_000_000，MAX_SURFACE_PIXELS=50_000_000；每跨页 100 层包含递归子层。DPI 仅接受 72、150、300。

- 本地浏览器预计工作集上限 768MiB，headless 2GiB；保守估算 12×最大源图像素＋12×输出面像素＋缓存实际字节＋128MiB。超预算明确拒绝并建议拆分或降规格，不暗降清晰度；不是实测内存保证。

- 同时解码／渲染 1 个输出面；图片缓存至多 128MiB；滤镜 tile 为 512×512，不创建整张 80MP 的浮点副本。每张 bitmap／canvas 在使用结束后释放。

- 所有层以左上为原点，x／y／width／height 为局部坐标，rotation 单位度；group 子层使用父坐标、组只允许统一缩放，避免首版引入不可表达的剪切。图片框缩放通过 crop／fitMode 表达；不支持的画布变换不能写回模型。

- fitMode='cover'|'contain'，默认 cover；cover 裁剪比例匹配图框，contain 明确留白；旋转保持源图方向归一化后的坐标。

- 保存后的 revision 为正整数；本地新项目 revision=1。保存用 expectedRevision 原子比较，成功加一；草稿的操作不擅自修改 revision。

- 自动保存空闲 2 秒、持续操作最多 10 秒触发；撤销至少 100 次操作，历史命令不含图片字节；本地保留最近 20 个恢复 revision，正在导出快照引用的资源不清理。

- C1 原创种子模板 5 个：婚礼、旅行、亲子、写真、国风各 1 个。它们是可编辑占位模板，不继承第三方图片或 Skill 输出。

- 关闭导出时释放所有临时资源；PNG 保留透明，JPEG 使用用户显式选择的底色（默认白色），JPG 质量 0.95。多面图片打 ZIP，PDF 每面一页。

共享类型由指定任务定义，任务外不要另起同义模型：

类型／端口	定义位置与最低字段			
DocumentModel, Layer, DocumentIssue, FilterRef, TemplateRef	任务 1；v0.1 示例为基础，known layers=image/text/shape/group；未知同版本层为 opaque 对象，保留全部 JSON；Issue 含 code、path、severity；FilterRef 含 presetId／recipeVersion／strength／seed，TemplateRef 含 id／version／sha256			
AssetRecord, FontRecord, ResourceManifest	任务 2；资源 id/version、文件 SHA-256、尺寸、来源／授权、状态；字体 face 可含多个本地 WOFF2 子集及各自 hash／字符覆盖			
ResourcePort	任务 2；read(id: string, variant: 'original'	'normalized'	'preview'	'font'): Promise<Uint8Array>；找不到则抛带 code 的错误，不能替代 variant
ResourceWriter, AssetBytes	任务 2 定义、任务 3 实现；putAsset(record:AssetRecord, data:AssetBytes):Promise<void>、putFont(record:FontRecord,files:ReadonlyMap<string, Uint8Array>):Promise<void>；AssetBytes 为 original／normalized／preview 三份字节；同 id 不同 hash 禁止覆盖			

类型／端口	定义位置与最低字段			
RevisionStore	任务 3；load(projectId: string, revision?: number): Promise<SavedRevision>；save(doc: DocumentModel, expectedRevision: number, resources: ResourceManifest): Promise<SaveResult>			
SavedRevision, SaveResult	任务 3；前者含 doc、docHash、resources、savedAt；后者为 {ok:true,value:SavedRevision} 或 {ok:false,code:'REVISION_CONFLICT'	'STORAGE_FULL'	'STORAGE_FAILED',recoveryDraft:DocumentModel}	
PixelTile, FilterRecipe	任务 4；tile 含 RGBA8 bytes、width、height、offsetX／Y、fullWidth／Height；FilterRef 从 document 导入			
RenderInput, RenderSurface, TextMetricsReport	任务 5；RenderInput 含 doc、surface:RenderSurface、manifest、port、signal、pixelScale；RenderSurface 含稳定 id、sourceSpreadId 或 cover、clip:{x,y,width,height}、bleedPx96、background、pixelWidth／Height；文字报告含 layerId、lines:string[]、overflow、unsupportedCodePoints			
ExportOptions, SurfaceSpec, ExportSnapshot	任务 8；options 含 format、dpi、view、pageIds、includeCover、bleedMm、jpegMatte；SurfaceSpec 扩展 RenderSurface 的物理 trim／bleed 元数据；snapshot 含已保存 revision、docHash、resources、options、optionsHash、createdAt			
PreflightReport	任务 8；含 snapshotHash、blockers、warnings、surfaces；每个 issue 有稳定 id，确认针对这些 id 和 snapshotHash，不接受全局“忽略全部”			
Artifact, ArtifactManifest	任务 9；文件名、MIME、bytes／SHA-256、surfaceId、像素／物理尺寸、DPI、revision、snapshotHash、renderBuild；不含原片 GPS			

文件结构

所有路径相对仓库根；每项任务列出具体文件，src/index.ts 仅导出公开接口。

workspace 包名固定为 @album/document、@album/assets、@album/storage、@album/render、@album/export、@album/web、@album/render-cli，依赖方向由上表决定，不跨包读取私有文件。

```
packages/document/src/          类型、校验、哈希、命令、模板
packages/assets/src/            导入、方向、字体、资源合同
```

packages/storage/src/	IndexedDB、revision、项目包
packages/render/src/	CPU 滤镜、字体布局、Fabric 映射
packages/export/src/	预检、快照、几何、编码与元数据
apps/web/src/	编辑器 UI、本地项目、导出面板
apps/render-cli/src/	可信本地快照渲染入口
tests/fixtures/	自制／明确授权的照片、字体、坏文档
tests/browser/	真实 Chromium 行为与导出检查
docs/implementation/c1/	实际环境、验收结果与导出样片

Task 1: 可校验、无损往返的文档模型

Files: Create package.json, package-lock.json, .nvmrc, tsconfig.json, vitest.config.ts, packages/document/package.json, packages/document/src/{model,validate,canonical,index}.ts, packages/document/src/document.test.ts, tests/fixtures/documents/minimal.json。

Interfaces: Consumes v0.1 JSON 字段及本计划 Global Constraints。Produces parseDocument(value: unknown): DocumentModel、validateDocument(doc: DocumentModel): DocumentIssue[]、canonicalDocument(doc: DocumentModel): string、hashDocument(doc: DocumentModel): Promise<string>。解析错误抛 DocumentValidationError，有 issues；未知同版本层不是解析错误，但返回 UNSUPPORTED_LAYER 导出阻断项。

☐ Step 1: 写失败用例 document.test.ts。复制 minimal fixture 后增加未知层和 advancedBlend:{mode:'multiply'}；断言 parse→canonical→parse 保留字段；相同对象不同键顺序 hash 相等，数组顺序不同 hash 不同。再断言 NaN、Infinity、负宽高、重复层 ID、6 层嵌套、循环对象、schemaVersion=99 分别产生明确错误；封面与内页 ID 全局唯一。（RF1）

```
expect(parseDocument(JSON.parse(canonicalDocument(doc)))).toEqual(doc);
expect(validateDocument(doc).map(i => i.code)).toContain('UNSUPPORTED_LAYER');
expect(() => parseDocument({...doc, schemaVersion: 99})).toThrow(DocumentValidationError);
```

☐ Step 2: 建立仅供测试运行的 workspace 配置，执行 npm run test:unit -- packages/document/src/document.test.ts。预期 FAIL：缺少 parseDocument 实现；若只有依赖／配置错误，先修复测试环境，不把它算作有效失败。配置与最小可运行模型同项提交。

☐ Step 3: 实现上述四个签名。已知字段严格校验、对象扩展字段透传；仅接收 JSON 值，禁止 prototype 污染键。canonical 键按 Unicode 码点排序、数组保持顺序、几何值保存为最多 6 位小数，hash 用 SHA-256。跨页及源图上限校验分开，未知层不能变成空层。生成 DocumentModel 的判别联合与 opaque 类型，不使用 Fabric JSON 作源数据。

☐ Step 4: 执行 npm run test:unit -- packages/document/src/document.test.ts 和 npm run typecheck，预期退出码均 0；记录 Node、npm、lockfile SHA 和兼容性结果。精确版本安装不兼容时停止扩展功能，依据最小复现选择受支持版本并记决策，不强制忽略 peer 依赖。

☐ Step 5: 提交 git add package.json package-lock.json .nvmrc tsconfig.json vitest.config.ts packages/document tests/fixtures/documents；git commit -m "feat(document): add lossless validated album model"。

Task 2: 不可变照片、EXIF 与本地字体

Files: Create packages/assets/package.json, packages/assets/src/{types,import-image,font-catalog,resource-port,index}.ts, packages/assets/src/assets.test.ts, scripts/{build-font-catalog,build-image-fixtures}.ts, tests/fixtures/images/{orientation-6,orientation-8,transparent,landscape,portrait}.json, tests/fixtures/images/README.md, apps/web/public/fonts/{manifest.json,OFL.txt}, tests/browser/assets.spec.ts, tests/browser/harness/{index.html,main.ts,host.ts}, playwright.config.ts。图像二进制由 fixture 脚本生成并加入同目录；字体子集文件由已锁依赖复制，禁止运行时下载字体。harness 只在测试启动 Vite 提供模块／fixture，后续浏览器核心测试复用；任务6开始另启动真实 web 做 UI 验证。

Interfaces: Consumes DocumentIssue。Produces资源共享类型和 ResourceWriter 接口、importImage(file: File, signal: AbortSignal): Promise<{record:AssetRecord,original:Uint8Array,normalized:Uint8Array,preview:Uint8Array}>、loadFont(record: FontRecord, port: ResourcePort, signal: AbortSignal): Promise<LoadedFont>、unsupportedGlyphs(text: string, font: LoadedFont): number[]。LoadedFont 含唯一 family、faces、codePoints、dispose()；字体集合内每个子集具有独立资源 id，可经 read(id,'font') 读取。

☐ Step 1: 写 assets.test.ts 与 assets.spec.ts。构造带四角色块的 EXIF 6／8 JPEG，断言归一化宽高交换、角块位置正确、original SHA 不变，重复加载不再次旋转；透明 PNG alpha 保留；25_000_001 字节、80_000_001 像素、伪 JPEG、坏 PNG、超长宽比但像素合法输入均有明确接受／拒绝结果。字体读失败／hash 不符／授权禁止使用必须失败；我们A´ 中缺字返回具体码点，不以系统 fallback 当成功。（RF3、RF4）

```
expect([result.record.width, result.record.height]).toEqual([200, 300]); // 源为300×200，EXIF6
expect(result.original).toEqual(originalBytes);
```

```
expect(unsupportedGlyphs('我们　　', font)).toContain(0x1f642);
```

☐ Step 2: 执行 npm run test:unit -- packages/assets/src/assets.test.ts 与 npm run test:browser -- tests/browser/assets.spec.ts，预期 FAIL：导入／字体入口未实现。

☐ Step 3: 实现三个函数与 ResourcePort。 检查 magic bytes 与图片头后解码；浏览器解码器统一应用 EXIF 一次，不再叠加手动旋转，归一化文件清除 EXIF orientation／GPS。保留原片字节独立存储；preview 长边 1600。seed 字体 Noto Sans SC／Noto Serif SC，各400／700真实字重，精确版本及 OFL 留档；fontkit 解析实际子集 cmap，按所需字符加载对应本地 faces，等待 FontFace.load 和 document.fonts.ready；family 用版本 hash 命名，不允许合成粗体冒充已有版本。abort 时释放 bitmap 和 ObjectURL。

☐ Step 4: 重跑上述两命令，预期全部 PASS。 记录 EXIF fixture 生成逻辑和颜色容差（JPEG 每通道 8）；核对 manifest 每份字体文件实际 SHA、来源与可用授权，禁止只读取 CSS unicode-range 当真实缺字证据。

☐ Step 5: 提交本项明确路径及新增 fixture、根依赖／lockfile；git commit -m "feat(assets): normalize imports and pin licensed fonts"。

Task 3: 原子保存、命令历史和恢复项目包

Files: Create packages/storage/package.json, packages/storage/src/{indexeddb-store,project-bundle,index}.ts, packages/document/src/commands.ts, apps/web/src/state/{history,autosave}.ts, tests/browser/storage.spec.ts。

Interfaces: Consumes文档哈希和 ResourceManifest。Produces IndexedDbRevisionStore implements RevisionStore, ResourcePort, ResourceWriter（构造参数 dbName:string）、applyCommand(doc: DocumentModel, command: DocumentCommand): DocumentModel、History(initial:DocumentModel)（execute(command:DocumentCommand)、undo()、redo() 均返回 DocumentModel）、Autosave(store:RevisionStore,getDraft:()=>DocumentModel,getResources:()=>ResourceManifest)（markDirty():void、flush():Promise<SaveResult>、dispose():void，status='dirty'|'saving'|'saved'|'conflict'|'error'）、exportProject(projectId:string,store:IndexedDbRevisionStore):Promise<Uint8Array>、importProject(bytes:Uint8Array,store:IndexedDbRevisionStore):Promise<SavedRevision>。DocumentCommand 为 insert/delete/update/reorder layer、add/delete/reorder spread、apply-template 的判别联合；update 只改已知允许字段，不替换整层对象。

☐ Step 1: 写 storage.spec.ts。 A／B 两标签页从 revision=1 保存，A 得 2，B 必须 REVISION_CONFLICT；注入事务 QuotaExceededError 后 load 仍返回 2、autosave.status='error'、恢复草稿存在，任务6再验证对应 UI 文案；用虚拟时钟验证 2 秒／10 秒；连续 100 次操作可全部撤销并重做；复制跨页后修改副本不改变原页；未知字段跨导出项目包／导入仍在。（RF1、RF2）

```
expect((await store.save(draftA, 1, resources)).ok).toBe(true);
expect(await store.save(draftB, 1, resources)).toMatchObject({ok:false,code:'REVISION_CONFLICT'});
expect((await store.load(projectId)).doc.revision).toBe(2);
```

☐ Step 2: 执行 npm run test:browser -- tests/browser/storage.spec.ts，预期 FAIL：仓库和 history 未实现。

☐ Step 3: 实现上述端口与类。 revision 元信息、文档、资源引用与 current 指针在同一 IndexedDB 事务更新；资源 blobs 按 hash 不可变，未挂接资源可稍后回收。提供 create(doc:DocumentModel,resources:ResourceManifest):Promise<SavedRevision建立 revision=1，不复用 save 的冲突语义；本地类再提供 pinResources(ownerId:string,manifest:ResourceManifest):Promise<void>／releaseResources(ownerId:string):Promise<void>，清理器跳过活动导出引用。项目包 ZIP 含版本 manifest、文档、原图／归一图／字体字节、SHA 与许可证；导入逐项验证，拒绝路径穿越、重复路径、超资源上限、解压后总量超过 1GiB 的包，不从包执行脚本或解析任意 URL。导入用新 projectId，校验完成后文档与资源在同一事务落库，保留 sourceProjectId／sourceRevision 追溯；配额不足保留原库，不先清空。冲突返回可另存草稿，不自动合并；清理 20 份以外历史时保留当前和正在导出引用。

☐ Step 4: 重跑命令并增加坏 ZIP／hash 错误导入断言，预期 PASS。 npm run typecheck 退出 0。首次浏览器未授权持久存储／不支持 IndexedDB 时显示具体错误和项目包恢复建议，不声称自动云备份。

☐ Step 5: 提交本项源文件、测试及依赖变更；git commit -m "feat(storage): add atomic revisions and recoverable local projects"。

Task 4: 33 个确定性 CPU 滤镜

Files: Create packages/render/package.json, packages/render/src/filters/{types,recipes,kernel,index}.ts, packages/render/src/filters/kernel.test.ts, scripts/build-filter-manifest.ts, packages/render/manifests/filters-v1.json；Read docs/ai-album-v0.1/filters.csv，不修改历史配方状态。

Interfaces: Consumes CSV 的 F01 – F33。Produces getRecipe(ref: FilterRef): FilterRecipe、filterTile(tile: PixelTile, recipe: FilterRecipe, strength: number, seed: number): Uint8ClampedArray；manifest 带 CSV SHA、kernelVersion='cpu-srgb-v1' 与操作版本。原图使用 null FilterRef，不能占用 F01。

☐ Step 1: 写 kernel.test.ts。 断言 33 个唯一连续 ID、顺序／默认强度 .65／seed1729 与 CSV 一致；strength=0 完全返回原字节；单 exposure=1、strength=1 时 [128,64,32,128]→[255,128,64,128]；相同 seed 同字节，改 seed 的 grain 结果变化；四块 tile 拼接等于整图；12 种操作均有至少一组数值 fixture，alpha 逐像素不变。

```
expect([...filterTile(tile, exposureOneRecipe, 1, 1729)]).toEqual([255,128,64,128]);
expect(filterTile(tile, recipe, 0, 1729)).toEqual(tile.bytes);
```

☐ Step 2: 执行 npm run test:unit -- packages/render/src/filters/kernel.test.ts，预期 FAIL：kernel／manifest 未实现。

☐ Step 3: 按附录 B 实现签名。 manifest 由 CSV 构建且可重复生成；CPU 为唯一标准路径，不引入 CSS／GPU 第二套效果。滤镜在归一化源图坐标上计算，再裁剪和缩放；preview 也由这一处理结果缩小。强度／参数越界拒绝，未知 recipeVersion 阻断，不自动使用最新版本。

☐ Step 4: 重跑测试并执行 npm run build:filters -- --check，预期 PASS／退出 0；生成自制色卡的 33 效果联系表供后续视觉评审。数值通过只证明确定性，不把 CSV 的 visual_review_required 自动改成视觉已验收。

☐ Step 5: 提交本项路径、联系表及依赖变更；git commit -m "feat(render): add versioned deterministic filter recipes"。

Task 5: 共用的画布、裁剪与中文文字布局

Files: Create packages/render/src/{scene-adapter,text-layout,image-pipeline,index}.ts, packages/render/src/text-layout.test.ts, tests/browser/render.spec.ts, tests/fixtures/documents/render-contract.json。

Interfaces: Consumes模型、资源端口、filterTile。Produces共享 RenderInput／RenderSurface／TextMetricsReport 类型、layoutText(layer: TextLayer, font: LoadedFont, measure: TextMeasure): TextMetricsReport、createScene(canvas: HTMLCanvasElement, input: RenderInput): Promise<SceneHandle>；handle 为 {update(input:RenderInput):Promise<void>, render():Promise<void>, dispose():void}。TextMeasure 为 (text:string,font:LoadedFont,fontSize:number,letterSpacing:number)=>number；渲染与预检共用同份文字报告。RenderSurface 在此定义，不依赖后续 export 包，编辑器预览构造同型 surface。

☐ Step 1: 写 text-layout.test.ts 和 render.spec.ts。 连续中文、中文标点、中英混排、换行符、A＋组合重音和 emoji grapheme 不被拆坏；行首不出现，。！？、）》】，行尾不出现（《【，放不下时报告 overflow；100px 高框、fontSize40／lineHeight1.4 的三行判为溢出。验证 cover／contain、组旋转、透明度和层序；并发 update 后迟到解码不能覆盖新页面；dispose 两次安全，ObjectURL／bitmap 创建释放计数平衡。（RF4、RF5）

```
expect(layoutText(threeLineLayer, font, measure).overflow).toBe(true);
expect(layoutText(chineseLayer, font, measure).lines.every(s => !/^[，。！？、）》】]/u.test(s))).toBe(true);
```

☐ Step 2: 执行 npm run test:unit -- packages/render/src/text-layout.test.ts 和 npm run test:browser -- tests/browser/render.spec.ts，预期 FAIL：共享 scene／布局未实现。

☐ Step 3: 实现三个入口。 Intl.Segmenter 按 grapheme 分段、显式标点禁则、Canvas measureText 使用已加载精确字体；lineHeight 为 fontSize 倍数、letterSpacing 为 px96。将已确定的文字行分别映射 FabricText，不让交互 Textbox 和导出再各自换行。框为布局边界，报告实际字形墨迹及逻辑行框越界，文字编辑用 HTML textarea overlay 后提交模型。图片先共享 CPU pipeline 再交由 Fabric；禁止 SVG 外链／可执行对象。update 采用代次令牌＋AbortSignal，旧异步结果必须丢弃并释放。

☐ Step 4: 重跑两命令，预期 PASS。 render-contract 使用相同输入重复渲染像素 hash 相等；不同输出倍率的几何比例相符；缺字体时 FAIL FONT_UNAVAILABLE，不截图系统字体作为预期。

☐ Step 5: 提交本项路径与依赖；git commit -m "feat(render): share crop and CJK scene rendering"。

Task 6: 可操作的网页图层与跨页编辑器

Files: Create apps/web/{package.json,index.html,vite.config.ts}, apps/web/src/{main,App}.tsx, apps/web/src/editor/{Editor,CanvasView,LayerPanel,PropertiesPanel,SpreadStrip,AssetPanel}.tsx, apps/web/src/styles.css, tests/browser/editor.spec.ts；Modify packages/document/src/commands.ts。

Interfaces: Consumes RevisionStore、History、Autosave、createScene。Produces Editor({initial,store,resources,port}:EditorProps):React.JSX.Element；editor 只经 DocumentCommand 更新模型，Fabric selection／拖动反馈转换成同一命令。应用层创建 store／端口，不在共享渲染中引用 React 状态。

☐ Step 1: 写 editor.spec.ts。 导入两张照片→拖入跨页→移动、裁剪、对齐、旋转→添加文字与矩形／圆／线→改变层序／显隐／锁定→组合与取消组合→撤销重做→保存刷新，断言模型与画面一致。两种字体及400／700粗细、字号／颜色／对齐／字距／行距可改；画框边界和中心吸附按6屏幕像素阈值工作；换图保持图框几何；删除使用中素材显示引用数量并阻止直接破坏引

用。删除跨页前确认数量；新增／复制／排序后封面不计入内页。StrictMode 双挂载后仍只有一个活跃画布监听器，切页无迟到图片污染。（RF5）

```
await page.getByRole('button', {name:'保存'}).click();
await expect(page.getByRole('status')).toHaveText('已保存至此浏览器');
await page.reload();
await expect(page.getByRole('button', {name:'跨页 1'})).toBeVisible();
```

☐ Step 2: 执行 npm run test:browser -- tests/browser/editor.spec.ts，预期 FAIL：编辑页与操作不存在。

☐ Step 3: 实现上述组件与命令。 页面三区为素材／画布／属性，底部跨页；1280px 完整布局，1024px 折叠侧栏。支持多选、复制、删除、上下层、6 个对齐方向、均匀分布、锁定、显隐；group 统一缩放和旋转，取消组合后绝对几何保持。吸附阈值为6屏幕像素、随缩放换算为逻辑距离，Alt 暂停吸附。双击裁剪，cover／contain 可切；换图保留框并重新计算合法 crop；导入逐项进度、失败重试、同 hash 重复提示，默认不新增重复素材。透明度 [0,1]，rotation 可规范化；命令校验失败不污染历史。离页有未保存提示，Ctrl/Cmd+Z 在输入框内优先原生文字撤销；空项目和资源错误显示操作路径。

☐ Step 4: 重跑测试及 npm run build、npm run typecheck，预期全部退出 0；实测键盘操作、1024／1280／1920 截图。没有可用真实浏览器时记录阻塞，不用静态 HTML 校验冒充交互通过。

☐ Step 5: 提交本项路径与依赖；git commit -m "feat(web): add editable multi-spread album workspace"。

Task 7: 原创模板与滤镜面板

Files: Create packages/document/src/{template,template.test}.ts, packages/document/templates/{wedding,travel,family,portrait,ink}-v1.json, apps/web/src/editor/{TemplatePanel,FilterPanel}.tsx, tests/browser/presets.spec.ts。

Interfaces: Consumes commands、filter manifest。Produces instantiateTemplate(template: TemplateVersion, bindings: Record<string,string>, idFactory:()=>string): Layer[]、applyTemplateCommand(spreadId:string,layers:Layer[],templateRef:TemplateRef):DocumentCommand；TemplateVersion 含 id／version、尺寸、slots／editableFields、licenseRef、原创来源；替换不删除素材仓库原片。

☐ Step 1: 写 template.test.ts 和 presets.spec.ts。 五类模板可筛选，预览不改文档；确认应用仅修改目标跨页，一次撤销恢复原层及引用；slot 不足／绑定素材不存在则明确失败，不截断照片。选中两张图片应用 F07、调整强度后一次撤销恢复两张；文字层不受滤镜影响；原图入口与 33 预设分开。模板复制层 ID 不冲突。

```
expect(history.undo()).toEqual(beforeTemplate);
const ids = instantiateTemplate(template, bindings, idFactory).map(l => l.id);
expect(new Set(ids).size).toBe(ids.length);
```

☐ Step 2: 执行 npm run test:unit -- packages/document/src/template.test.ts 和 npm run test:browser -- tests/browser/presets.spec.ts，预期 FAIL。

☐ Step 3: 实现签名与面板。 seed 模板只含原创几何、独立标题／日期字段、带 hash 的模板版本；不含购买的假授权标识或受限示例。模板单次作用于当前跨页，整本智能分配留在 C2；待选照片保持可见清单，不声称已自动全部入册。配方参数直接引用 manifest，不复制成 UI 私有值。

☐ Step 4: 重跑两命令，预期 PASS；核对 5 模板来源记录及 33 项原图／强度界面，联系表保留“ 待视觉确认 ”标记。

☐ Step 5: 提交本项路径；git commit -m "feat(editor): add original templates and filter controls"。

Task 8: 冻结快照、导出几何与阻断预检

Files: Create packages/export/package.json, packages/export/src/{types,geometry,snapshot,preflight,index}.ts, packages/export/src/preflight.test.ts, tests/browser/preflight.spec.ts, apps/web/src/export/PreflightPanel.tsx。

Interfaces: Consumes SavedRevision、ResourcePort、layoutText。Produces freezeSnapshot(saved:SavedRevision,options:ExportOptions,createdAt:string):Promise<ExportSnapshot>、planSurfaces(snapshot:ExportSnapshot):SurfaceSpec[]、preflight(snapshot:ExportSnapshot,port:ResourcePort,signal:AbortSignal):Promise<PreflightReport>、authorizeExport(snapshot:ExportSnapshot,report:PreflightReport,acknowledgedWarningIds:string[]):void。冻结后深拷贝且只读；snapshotHash 绑定 docHash、资源 manifest、optionsHash 和创建时间。

☐ Step 1: 写 preflight.test.ts 与 preflight.spec.ts。 原图 read 失败但 preview 存在仍 ASSET_MISSING；未知层、字体／授权失败、缺字、文字溢出、超像素预算各阻断；EXIF 后裁剪有效 PPI 与预期一致，199／100 警告、99 强警告，未确认不得导出。600×300mm、300DPI、0／3mm、view=spread／includeCover=false 分别断言 7087×3543／7157×3614。另用带封面的 fixture 验证

单页导出顺序 front→内页1…N→back，spread 模式 front→跨页1…N→back；cover 各只占单页。snapshot=12 后保存13，快照仍12；修改 DPI 后旧报告无效。（RF1、RF3、RF4）

```
expect(planSurfaces(snapshotWith3mm)[0]).toMatchObject({pixelWidth:7157,pixelHeight:3614});
expect(report.blockers.map(i => i.code)).toContain('ASSET_MISSING');
expect(() => authorizeExport(snapshot, warningReport, [])).toThrow('WARNINGS_UNACKNOWLEDGED');
```

☐ Step 2: 执行 npm run test:unit -- packages/export/src/preflight.test.ts 和 npm run test:browser -- tests/browser/preflight.spec.ts，预期 FAIL。

☐ Step 3: 实现四个入口。 从已保存记录构造快照，源图／字体读取核对 SHA；图片以 min(可用裁剪源宽÷绘制宽英寸,可用裁剪源高÷绘制高英寸) 算 PPI，contain 使用实际绘制区域，group 缩放纳入。旋转图片按未旋转框对应源尺寸算，不用旋转后外接框夸大 PPI。PNG／PDF 像素预算独立检查；几何只最终输出 round，出血扩展输出面，超出裁切区的图像原样延展、没有内容处用背景，不拉伸整幅页面冒充出血。资源预计超内存亦阻断。pageIds 取稳定逻辑页 ID（如 spr_01:left／right），spread 视图使用 spr_01；先按文档顺序生成输出面，再筛选所选 ID，拒绝与 view 不匹配或不存在的 ID。

☐ Step 4: 重跑两命令，预期 PASS。 预检面板逐项显示阻断／警告、实际像素和快照 revision；安全区 5mm／中缝关键文字警告。C1 输入若带尚未实现的 assignmentManifest 校验版本则阻断 UNSUPPORTED_LAYOUT_MANIFEST，C2 再接入重新派生照片清单的验证器；不把未知 manifest 当通过。含未支持的 AI 来源／标识要求时明确 UNSUPPORTED_PROVENANCE，C5 接入前不能忽略。为这两个拒绝行为各增加一条断言。

☐ Step 5: 提交本项路径与依赖；git commit -m "feat(export): freeze revisions and enforce preflight"。

Task 9: 实际 PNG／JPEG／PDF 编码与下载

Files: Create packages/export/src/{encode-png,encode-jpeg,encode-pdf,artifact-manifest,run-export}.ts, packages/export/src/metadata.test.ts, apps/web/src/export/ExportPanel.tsx, tests/browser/export.spec.ts。

Interfaces: Consumes ExportSnapshot、PreflightReport、authorizeExport、createScene。Produces runExport(snapshot:ExportSnapshot,report:PreflightReport,ack:string[],port:ResourcePort,signal:AbortSignal,onProgress:(done:number,total:number)=>void):AsyncGenerator<Artifact>、encodePng(canvas:HTMLCanvasElement,dpi:number):Promise<Uint8Array>、encodeJpeg(canvas:HTMLCanvasElement,dpi:number,matte:string):Promise<Uint8Array>、encodePdf(pages:EncodedPage[],snapshot:ExportSnapshot):Promise<Uint8Array>；EncodedPage 含 surface、mime、bytes。PDF 首期页面逐面渲染，累计编码数据达到 256MiB 时显式拒绝并建议拆分，不无界保留原始像素。

☐ Step 1: 写 metadata.test.ts 和 export.spec.ts。 校验 PNG pHYs、JPEG JFIF densityUnits=1／Xdensity=Ydensity=300；分辨率取 pHYs 反算容差 .02DPI，不能只读文件名。PDF pageCount、MediaBox、TrimBox／BleedBox 与物理尺寸一致；多面 ZIP 名称顺序 0001-front、0002-spread-01 等由 SurfaceSpec 固定；透明 PNG alpha 保留，JPEG 透明区用指定白色。编辑到13后导出快照12，标题和像素仍12；中途 cancel 不提供完整成功下载，资源计数归零。（RF5）

```
expect(decodedPng.width).toBe(7157);
expect(jfif).toMatchObject({densityUnits:1,xDensity:300,yDensity:300});
expect(pdf.getPageCount()).toBe(6);  //  4个内页＋封面正背面
```

☐ Step 2: 执行 npm run test:unit -- packages/export/src/metadata.test.ts 和 npm run test:browser -- tests/browser/export.spec.ts，预期 FAIL。

☐ Step 3: 实现编码与生成器。 PNG 插入／替换 pHYs 并重算 CRC；JPEG 写入／规范 JFIF 密度，不保留源 EXIF；PDF 使用 pdf-lib，MediaBox 包含出血、TrimBox 为裁切范围、BleedBox=MediaBox，宽高用 mm×72/25.4，按用户视图逐面一页。先 authorizeExport，再依序渲染／编码；每面完成释放 canvas。网页在调用前以 snapshotHash 执行 store.pinResources，finally 执行 releaseResources；CLI 的只读包天然冻结。ArtifactManifest 含实际文件 SHA、revision、资源 hash、renderBuild，不写用户位置／GPS。ZIP 使用 fflate 流式条目，累积下载超过 256MiB 提示拆分；浏览器不伪装持久后台队列。

☐ Step 4: 重跑两命令，预期 PASS；从导出字节重新解码检验尺寸、色块／文字截图和元数据，不只检验 mock。下载 PNG／JPG／PDF 与 JSON 清单作为程序样片候选；JPEG 视觉比较容许有损误差，不要求与 PNG 字节一致。

☐ Step 5: 提交本项路径与依赖；git commit -m "feat(export): encode real image and PDF artifacts"。

Task 10: 固定 Chromium 渲染入口、端到端验收与交付

Files: Create apps/render-cli/{package.json,src/main.ts,src/host.ts}, apps/web/src/render-entry.ts, tests/browser/c1-acceptance.spec.ts, tests/fixtures/documents/acceptance.json, scripts/verify-artifacts.ts, .github/workflows/c1.yml, docs/implementation/c1/{README.md,environment.json,acceptance-results.json}, docs/export-samples/program-c1/README.md；Modify 根 README.md、package.json。

Interfaces: Consumes共享 render／export，不重写 scene。Produces CLI npm run render:snapshot -- --bundle <project.zip--snapshot <snapshot.json--output <directory>（仅可信开发／worker 调用），成功退出0，参数／预检错误2，资源／执行失败3。浏览器 render-entry 暴露 renderSnapshot(request:TrustedRenderRequest):Promise<ArtifactManifest>，请求仅携带快照和 manifest 内资源 ID，不接受任意 URL 或代码。

☐ Step 1: 写 c1-acceptance.spec.ts。 从空浏览器开始导入授权照片→应用模板→编辑中文→设置滤镜→保存→项目包恢复→预检→导出三格式；同一 fixture 在网页和固定 Chromium render-entry 中输出 PNG 解码像素 hash 相同。HEADLESS 请求外部 HTTP、file://、../ 路径、未知资源 ID 被拒；50MP 边界与低内存预算有可重复拒绝结果，不能悄悄缩小。重复运行同快照像素一致，metadata／文件排序一致；ZIP 时间等容器字节不作为跨系统逐字节承诺。

```
expect(webPixelsHash).toBe(headlessPixelsHash);
expect(artifactManifest.revision).toBe(12);
expect(artifactManifest.snapshotHash).toBe(snapshotHash);
```

☐ Step 2: 执行 npm run test:browser -- tests/browser/c1-acceptance.spec.ts，预期 FAIL：CLI／端到端入口未接通。

☐ Step 3: 实现入口与验证脚本。 host 仅绑定 127.0.0.1 随机端口、每次随机 token，资源查表无任意路径；Playwright 拦截出站请求，仅允许本次受信 host／blob，固定 Chromium 版本、locale=zh-CN、timezone=UTC、DPR=1，超时120秒，关闭后清理临时目录。它不作为公网 API 发布。CI npm ci→npx playwright install --with-deps chromium→typecheck／unit／build／browser；记录 lockfile SHA、浏览器实际版本、OS、字体／kernel／build hash。无凭据也能运行 C1，不连微信／支付／AI。

☐ Step 4: 执行最终验证 npm ci、npm run typecheck、npm run test:unit、npm run build、npm run test:browser、npm run verify:artifacts -- docs/export-samples/program-c1，预期全部退出0。样片至少含600×300mm、300DPI、3mm出血的 PNG／JPG、4个内页＋独立封面正背面的单页PDF、快照／预检／资源manifest／实际环境与 SHA。记录耗时和峰值内存观察值，不把它升级为上线 SLA。按选择的执行方式安排独立复核，修复阻断问题后再认定 C1 完成。

☐ Step 5: 提交本项路径、实际程序样片与验收记录；git commit -m "feat(c1): verify reproducible editor-to-export delivery"。根 README 改成实际可运行入口，保留历史 AI 样片标注；推送实施分支、创建草稿实现 PR，不能直接宣称生产上线。

附录 A：依赖、命令与验证状态

官方 npm registry 元数据于 2026-10-21 查阅；以下为计划候选精确版本，尚未安装验证兼容性，不能把这张表当已生成 lockfile 或安全审计。

依赖	候选版本	许可元数据／用途
node／npm	24.19.0／11.9.0	当前执行环境；Node 24 LTS 系列
react／react-dom	19.3.1／19.3.1	MIT；编辑界面
vite／@vitejs/plugin-react	8.3.5／6.1.2	MIT；构建
typescript	7.0.2	Apache-2.0；类型检查
fabric	7.4.0	MIT；仅作为 adapter
vitest	5.0.5	MIT；纯逻辑测试
@playwright/test	1.64.1	Apache-2.0；绑定其浏览器修订版本
zod	4.6.7	MIT；严格已知字段＋透传扩展字段
idb	8.0.4	ISC；本地事务封装
pdf-lib／fflate	1.17.1／0.8.3	MIT；普通 PDF／ZIP
@fontsource/noto-sans-sc	5.3.1	OFL-1.1；本地可追溯字体子集
@fontsource/noto-serif-sc	5.3.1	OFL-1.1；第二字体、400／700真实字重
fontkit	2.0.4	MIT；读取真实字体 cmap／度量；浏览器兼容在任务2验证
tsx	4.23.15	MIT；可信 CLI 与构建脚本

依赖	候选版本	许可元数据／用途
@types/node／@types/react／@types/react-dom	24.19.2／19.3.0／19.3.0	MIT；类型版本对齐

新增依赖先说明用途，记录实际 LICENSE 与传递依赖，不无目的引入 Zustand／完整后端框架／图片生成 SDK。当前 C1 不需要数据库、Redis 或付款账号。

根脚本定义：test:unit=vitest run；typecheck=tsc --noEmit（项目引用可分包但总命令不变）；test:browser=playwright test；dev=vite --config apps/web/vite.config.ts；build 构建共享包／web／CLI；build:filters=tsx scripts/build-filter-manifest.ts；render:snapshot=tsx apps/render-cli/src/main.ts；verify:artifacts=tsx scripts/verify-artifacts.ts。脚本随其拥有任务落地，计划阶段不创建空实现来伪造通过。

附录 B：CPU 滤镜 v1 算法合同

这是对 v0.1 留待固定算法的本次工程建议，需与计划一起评审；所有操作按 CSV 顺序执行。RGB 归一化至 [0,1]，每步 clamp，内部 float64，最后量化 round(255×C)；alpha 原字节不变。亮度 Y=.2126R+.7152G+.0722B。

操作	定义
exposure:e	clamp(C×2^e)
contrast:k	clamp((C-.5)×(1+k)+.5)
saturation:s	clamp(Y+(C-Y)×(1+s))
temperature:t	clamp([R+.08t,G+.02t,B-.08t])
tint:q	clamp([R+.04q,G-.08q,B+.04q])
gamma:g	C^(1/g)，g>0
grayscale:a	C×(1-a)+Y×a
sepia:a	标准矩阵行 [.393,.769,.189]、[.349,.686,.168]、[.272,.534,.131] 后 clamp，再与输入按 a 混合
duotone:dark,light	dark×(1-Y)+light×Y，颜色由十六进制转归一化 sRGB
posterize:L	round(C×(L-1))/(L-1)，L 为2 – 256整数
vignette:v	源图像素中心归一化到 x,y∈[-1,1]；d=min(1,sqrt(x ² +y ²)/sqrt(2))；RGB乘 1-v×clamp((d-.2)/.8)^2
grain:a	源整数坐标 x,y 与 seed 混合：h=seed XOR imul(x+1,73856093) XOR imul(y+1,19349663)；MurmurHash3 fmix32 后转无符号 n=h/4294967296-.5；RGB各通道加 a×n

fmix32：h ^= h>>>16；h=imul(h,0x85ebca6b)；h ^= h>>>13；h=imul(h,0xc2b2ae35)；h ^= h>>>16。grain 在同像素使用同一个噪声值；tile 坐标必须加 offset，因此切块不改变颗粒。

最终强度 s 使用 IEC sRGB 转线性函数（阈值 .04045，低段除12.92，高段 ((C+.055)/1.055)^2.4），混合 linear(original)×(1-s)+linear(effect)×s，再转回 sRGB（阈值 .0031308，低段乘12.92，高段 1.055×L^(1/2.4)-.055）。s=0 必须字节直通。不得把 s 乘进每个配方参数来冒充此混合。

自审与验收归属

规格／风险	本计划对应项	说明
文档、方向、资源不可变与未知字段	1／2／3	RF1 – RF3 有指定失败用例
33滤镜、原图、强度与多选撤销	4／7	数值确定性与视觉确认分开
网页图层、文字、形状、模板、跨页、恢复	3／5／6／7	C1 完整基础编辑；C2 承担整本自动布局
V3-EXPORT-01／02／03	2／5／8／9	缺原图阻断、中文／字体／溢出、像素与元数据
V3-EXPORT-04	3／8／9／10	C1 验证冻结后继续编辑；持久队列／事务在 C4
RF4／RF5	2／5／6／8／9	字体竞态、StrictMode、取消和资源生命周期
小程序、登录、会员、队列、AI 与运营	路线 C3／C4／C5	明确后续验收，不作为 C1 已完成项

自审结论：10 项任务的接口名按统一类型表引用，失败／通过命令与文件归属齐全；没有把技术候选、计划用例或现有调研结果记作产品通过证据。下一步是评审本计划并确认 AI 执行方式，然后从任务1开始。

全程 AI 开发：交付顺序与上线条件

执行依据

用户：全程 AI 开发。范围：v0.3 基线及其引用的 v0.1 商业、测试运维规则。研发按可验收单元推进，暂不根据虚构团队人数估算人周，也不把计划任务数换算为交期。

当前完成：需求与原型、定向调研和算法验证、C1 – C6共44项详细实施计划及全模块接口／数据合同。当前未完成：产品代码、真实供应商集成、部署、支付联调和上线验收。

配置需求补充见 v0.4 配置中心与素材目录：C3 – C5计划编制时需纳入配置版本、云存储根前缀／用户目录、AI模型与费用、20类配置及对应验收。此补充为待评审方案，未使C1自动包含云端实现。

交付顺序

单元	AI 实施结果	验收关口
C1 文档与渲染	网页导入、完整基础图层编辑、33 滤镜、原创模板、保存恢复、预检、实际 PNG／JPG／PDF；共享 headless 渲染入口	同一冻结文档在网页与固定 Chromium 中一致；中文不漏字；缺原图阻断；输出像素、页序和元数据正确
C2 排版与完整性	整本照片分配、跨页候选、照片用途与 assignmentManifest、锁定与局部重排、核对页	12 输入不能只放 11；50 张／10 跨页／最多 2 张报告无解；过期候选不能覆盖修改
C3 账号、素材与跨端	模块化 API、PostgreSQL、微信身份、工作区、私有素材上传、权限、云端 revision、小程序快速成册	跨工作区访问被拒；微信身份不误合并；小程序 patch 保留高级字段；网页／小程序冲突有恢复路径
C4 任务与商业	Outbox＋持久化任务＋导出 worker、租约和取消、会员、AI 点数账本、兑换码、支付与退款	Redis 重建不丢任务；重复回调不重复发放；重复结算不重复扣点；取消与迟到结果竞态可重放
C5 AI 与运营	自有／可商用预设、水墨彩绘、参考图用途、报价与生成、审核／标识、素材入册、运营后台	超时先查询不重复付费提交；失败释放点数一次；许可不明预设不可上架；生成图与可编辑标题分层
发布验收	预发环境、备份恢复、告警、权限回归、真实设备验证、导出样本归档、发布／回滚记录	测试运维手册要求与上述各单元验收全部有证据；没有凭据或未联调的能力不得标为上线可用

每单元在实施前补齐独立详细计划并核对前一单元接口。首期小程序仍包含在 C3，会员与队列仍包含在 C4，AI 水墨彩绘仍包含在 C5；C1 是第一个可演示单元，不能作为完整首发验收。

AI 任务执行规则

1. 先读取当前规格、计划与实际代码，再写一个有明确断言的失败用例。

2. 确认失败原因，再实现该项功能；运行针对性检查后提交。

3. 更新任务状态、实际验证环境、命令结果与限制；失败保留记录并修复，不用文案代替验证。

4. 遇到改变用户范围、商业费用、供应商授权或生产发布的决定，提供具体候选与影响再确认。常规可逆研发决定由 AI 执行。

5. 单元结束做接口、权限和功能复核；独立复核方式按用户选择的执行方法落实。接入真实服务前先完成模拟契约和可审查实现。

真实接入前需要的信息

时点	一次收集的内容	未提供前可继续的工作
C3 微信联调前	主体类型、网页／小程序 AppID、可用登录能力、服务域名与平台配置；密钥通过秘密配置注入	API、测试身份适配器、授权测试、小程序本地流程
C4 支付联调前	商户号、渠道准入、真实 SKU／价格／有效期／点数规则、退款和权益处理规则；小程序平台限制按当期官方规则核验	订单状态机、模拟支付回调、账本和重复通知测试；不预设小程序可直接购买全部数字权益
C5 供应商接入前	模型候选、服务条款和商业授权、单次成本／额度、数据用途与保留政策、标识／审核能力	统一供应商契约、可重放模拟结果、费用幂等测试、原创预设登记
预发／生产前	云区域、域名、存储、数据库、备份、运维告警接收方式、平台资质与发布主体	可部署容器、环境模板、恢复演练脚本、成本预算表和待核验清单

不把密钥写入文档、Git 或聊天示例。确认事项只在其依赖真正到来时提出，当前 C1 不依赖这些真实账号。

本次计划新增的工程建议

C1 采用本地单用户存储，作为云端项目仓库的端口实现；明确显示“ 仅此浏览器保存 ”。浏览器本地导出用于演示与恢复验证；可信本地 headless 入口验证未来 worker 的共享渲染能力。持久化导出队列、登录、云备份、会员校验由 C3／C4 接管。这些是待评审实施边界，不是对商业首发范围的删减。

基线要求的归属

要求	主验收单元	C1 铺垫
V3-MODE-01	C2／C5	普通编辑不调用生成、不扣点
V3-LAYOUT-01／02／03	C2	保留排版扩展字段；尚不接受自动布局候选
V3-EXPORT-01／02／03／04	C1＋C4	C1 实现资源校验、中文、尺寸与不可变快照；C4 验证排队和权益事务
V3-ART-01	C5	图片与文字层独立；C1 不生图
V3-LICENSE-01	C5	C1 仅提供自制模板与可商用字体及依赖清单
V3-CROSS-01	C3	C1 保存未知字段，未知层阻断渲染；C3 再验证服务端 patch 白名单

评审通过后锁定对应计划版本；改变接口或验收值时记录原因与受影响用例，避免只改界面而遗留旧合同。